

ИНДЕКСЫ

Индексы — это специальные структуры данных, ускоряющие доступ к строкам таблиц по значениям столбцов. В PostgreSQL они особенно полезны для WHERE, JOIN, ORDER BY, GROUP BY и агрегаций. Есть разные типы: B-Tree, Hash, GiST, GIN и др. Индексы ускоряют выборки, но замедляют вставки/обновления/удаления.

Индекс представляет собой отсортированный список значений одного или нескольких столбцов, с указателями на соответствующие строки в таблице.

Индекс:

- Упорядочен (например, по возрастанию чисел или алфавиту);
- Ссылается на строки в таблице;
- Автоматически используется СУБД при выполнении запроса (если сочтет нужным).

Индекс применяется:

- При **фильтрации** (WHERE): WHERE StudID = 18;
- При **соединении таблиц** (JOIN ON): ON Students.GroupID = Groups.ID;
- При **агрегации** (MIN, MAX) — если по индексируемому столбцу;
- При **сортировке** (ORDER BY);
- При **группировке** (GROUP BY);
- При **поиске по шаблону**, если шаблон начинается с фиксированных символов (например, 'abc%', но не '%abc').

ТИПЫ ИНДЕКСОВ

1. B-tree (по умолчанию)

Стандартный тип. Используется для сравнения (=, <, >, BETWEEN, ORDER BY, LIKE 'abc%').

2. Hash-индекс

Для точных сравнений =. Не поддерживает диапазоны.

3. GiST (Generalized Search Tree)

Для гео-данных, полнотекстового поиска, вложенных диапазонов и пр.

4. GIN (Generalized Inverted Index)

Для массивов, JSONB, документов, полнотекстового поиска (@@, tsvector).

5. SP-GiST

Для иерархий, IP-сетей, кластеров (быстрый поиск в редких структурах).

6. BRIN (Block Range Index)

Для очень больших таблиц, где значения идут по порядку. Мал по размеру, но ограничен.

Когда индексы неэффективны

- **Таблица маленькая** — проще просканировать всю таблицу.
- **Выборка большая** (например, WHERE age > 0) — индекс не даст выгоды.
- **Шаблон начинается с %** — индекс LIKE '%abc' не используется.
- **После сильных обновлений** — индекс может стать "грязным", и СУБД выберет полный перебор.

Когда СУБД сама не использует индекс

- Если статистика считает, что выгоднее полный перебор;
- Если WHERE содержит OR с неиндексируемыми условиями;
- Если соединение требует материализации и сортировки вне индекса.

Индексы и план выполнения запроса

СУБД:

1. Парсит запрос;
2. Строит дерево;
3. Смотрит на доступные индексы;
4. Выбирает **оптимальный план** (по оценке стоимости: I/O, CPU, объем вывода);
5. Может использовать индекс или проигнорировать его.

ПОДРОБНО ПРО КАЖДЫЙ ТИП

1. B-tree (по умолчанию)

Структура:

- Сбалансированное дерево поиска (B+Tree), где:
 - **Листовые узлы** содержат ключи и ссылки на строки таблицы;
 - **Внутренние узлы** — только ключи и указатели на дочерние узлы.

Работа:

- Данные **отсортированы**.
- Поиск осуществляется **двоичным спуском** от корня к листу (логарифмическая сложность).
- Листья связаны **двусвязным списком** — быстрая сортировка и сканирование по диапазону.

Поддерживает:

- =, <, >, BETWEEN, ORDER BY, LIKE 'abc%'
- может использоваться для DISTINCT, MIN(), MAX()

2. Hash-индекс

Структура:

- Хеш-таблица, в которой значения прогоняются через хеш-функцию, и результат указывает на сегмент ссылки на строки.

Работа:

- Быстрый поиск по точному совпадению =.
- Распределение по **хеш-бакетам**.
- **Не поддерживает** диапазонные операции.

Ограничения:

- Раньше (до PostgreSQL 10) не поддерживались транзакции и журнал WAL.
- Работает только с =, не может быть использован в ORDER BY.

3. GiST (Generalized Search Tree)



Структура:

- **Обобщенное дерево поиска**, гибко определяемое пользователем.
- Хранит не конкретные значения, а **приближения (bounding boxes, диапазоны)**.



Работа:

- Используется в гео-пространственных запросах, полнотекстовом поиске, вложенных диапазонах.
- Обеспечивает **приблизительный поиск** с возможностью уточнения на втором шаге.



Поддерживает:

- @>, <<, && и др. — например, для интервалов, координат, IP-диапазонов.

4. GIN (Generalized Inverted Index)



Структура:

- **Инвертированный индекс**: мапа «значение → список документов/строк», как в поисковых системах.



Работа:

- Очень эффективен для:
 - массивов (int[])
 - JSONB (@>, ?)
 - полнотекста (tsvector)
- Позволяет быстро находить все строки, где значение **входит в массив, JSON или текст**.



Поддерживает:

- @@, @>, ?, ?|, ?&, ~, и др.

5. SP-GiST (Space-partitioned GiST)



Структура:

- Делит пространство на части (например, квадранты).
- Подходит для **нераспределённых, иерархических структур** (IP-сети, дерево вложенности).



Работа:

- Использует **жесткую** структуру для разбиения значений (например, префикс дерева).
- Эффективен для **поиска ближайших соседей** и маршрутизации.

6. BRIN (Block Range Index)



Структура:

- Хранит **сводную информацию о блоках строк** (например, минимальное и максимальное значение).



Работа:

- Не индексирует каждую строку.
- СУБД проверяет, попадает ли условие запроса в **диапазон значений блока**.



Подходит только, если:

- Таблица **огромная**;
- Данные **отсортированы по индексу**.

СРАВНИТЕЛЬНАЯ ТАБЛИЦА

Тип индекса	Структура	Лучше всего для	Поддерживает	Особенности
B-tree	B+Tree	=, <, >, диапазоны	ORDER BY, WHERE	По умолчанию
Hash	Хеш-таблица	Только =	WHERE	Очень быстрый =, но ограничен
GiST	Обобщённое дерево	Гео, диапазоны	@>, && и т.п.	Приблизительный поиск
GIN	Инверт. индекс	Массивы, JSONB, текст	@>, @@, ? и др.	Идеален для contains и fulltext
SP-GiST	Пространственное	IP, маршруты	<<= и пр.	Иерархии, маршруты
BRIN	Блочный	Очень большие таблицы	Диапазоны	Маленький, но грубый

ОПТИМИЗАЦИЯ

Оптимизация SQL-запросов — это выбор наилучшего **плана выполнения**, минимизирующего ресурсы (I/O, CPU, память). В PostgreSQL она реализуется через несколько этапов: парсинг, преобразование, планирование и исполнение. СУБД учитывает законы реляционной алгебры, статистику таблиц и наличие индексов. Хорошая оптимизация = меньше чтения, меньше записей, больше конвейерной обработки.

Общий процесс выполнения запроса в PostgreSQL

1. **Парсинг (parser):**
Синтаксический разбор → дерево запроса.
2. **Переписывание (rewriter):**
Обработка представлений, правил, привилегий.
3. **Планировщик и оптимизатор (planner):**
 - Построение **всех возможных планов** выполнения.
 - Выбор **наиболее эффективного** (по стоимости).
4. **Исполнение (executor):**
 - Пошаговое выполнение плана — с JOIN, фильтрацией, сортировкой и т.д.

Как оптимизатор выбирает план

Основной критерий: оценочная стоимость плана

Она учитывает:

- 📦 **Число обращений к внешней памяти;**
- ⌚ **Время чтения/записи;**
- 💾 **Размеры промежуточных данных.**

Чем меньше «стоимость» — тем лучше.

Законы, которые применяет оптимизатор (реляционная алгебра)

1. **Коммутативность соединений**
 $R \bowtie \theta S \equiv S \bowtie \theta R$
2. **Ассоциативность**
 $R \bowtie \theta (S \bowtie \phi T) \equiv (R \bowtie \theta S) \bowtie \phi T$
3. **Выборка может быть вынесена "вниз"**
 $\sigma \phi (R \bowtie S) \equiv \sigma \phi (R) \bowtie S$ (если ϕ касается только R)
4. **Двойная выборка = вложенные фильтры**
 $\sigma \theta \wedge \phi (R) \equiv \sigma \theta (\sigma \phi (R))$
5. **Проекция может быть применена к R до соединения**
 $\pi_A (R \bowtie \theta S) \equiv \pi_A (\pi_A (A \cup B) (R) \bowtie \theta S)$

Стратегии оптимизации

- Делать выборку раньше (сначала $\sigma_{A.type='X'}(A)$, потом JOIN)
 - `SELECT * FROM A JOIN B ON A.id = B.aid WHERE A.type = 'X';`
- Делать проекцию раньше (убрать ненужные поля до соединения)
 - `SELECT name FROM STUDENTS JOIN GROUPS ON ...;`
- Использовать **левосторонние деревья, планы с левой ассоциативностью:**
 - позволяют избежать **материализации**
 - можно обрабатывать **конвейером**

Конвейерная обработка vs материализация

Конвейер

- Результат одной операции **сразу** подается следующей
- Минимум памяти, минимум времени

Материализация

- Сохраняются промежуточные результаты
- Затраты на **запись/чтение**
- Может потребоваться при DISTINCT, GROUP BY, ORDER BY

Виды JOIN-ов и их стоимость

1. Nested Loop Join

- Простая реализация
- Хорошо при малом числе строк (особенно во внутренней таблице)
- Плохо, когда обе таблицы большие и без индекса

для каждой строки r из R :
 для каждой строки s из S :
 если r и s удовлетворяют условию соединения:
 вернуть $r + s$

2. Hash Join

- Создает хеш-таблицу по одной таблице (в памяти)
- Идеален для = соединений, плох в остальном
- Операция требует много памяти

Создать хеш-таблицу по S : $key \rightarrow$ список строк
 для каждой строки r из R :
 найти в хеш-таблице строки s , где $s.key = r.key$
 для каждой найденной пары (r, s) :
 вернуть $r + s$

3. Sort-Merge Join

- Сортирует обе таблицы $\rightarrow$ сливает
- Лучше при уже отсортированных данных или больших таблицах
- Требуется **материализацию**
- Требуется сортировка на лету (если нет индекса или кластеризации)

Отсортировать R по key
 Отсортировать S по key
 Пока не достигнут конец обоих списков:
 сравнить $r.key$ и $s.key$
 если равны – вернуть комбинацию (может быть несколько вариантов)
 иначе – продвинуться по меньшему

Индексы в оптимизации

Индексы особенно полезны:

- Для WHERE, JOIN, ORDER BY, GROUP BY
- При ограниченных выборках

Но: могут ухудшить производительность, если:

- выборка большая
- данные не селективны
- таблица маленькая

Планировщик может **не использовать** индекс, даже если он есть!

JOIN тип	Поддержка =	Подходит для больших таблиц	Требуется индексов	Использует сортировку	Использует хеш
Nested Loop	✓	✗ (если без индекса)	✓ желательно	✗	✗
Hash Join	✓	✓	✗	✗	✓
Sort-Merge Join	✓ /диапазон	✓	✗ (но желательно)	✓	✗

ПЛАН ВЫПОЛНЕНИЯ ЗАПРОСА

План выполнения запроса в PostgreSQL — это детальный алгоритм, по которому СУБД действительно *выполняет* SQL. Он создается автоматически планировщиком (planner) и зависит от структуры запроса, статистики таблиц, индексов и допустимых стратегий. PostgreSQL может сгенерировать десятки/сотни альтернативных планов, и выберет самый дешевый по оценочной модели.

Что такое план выполнения запроса

SQL — **декларативный** язык: ты говоришь, *что* хочешь получить, а не *как*.

СУБД строит **программу** (query plan), которая:

1. Выбирает порядок операций (фильтрация, проекция, соединение)
2. Выбирает физические алгоритмы (Index Scan, Hash Join и т.д.)
3. Оценивает **стоимость** каждого варианта
4. Выбирает оптимальный (наименее затратный)

Этапы построения плана

1. **Парсинг (parser)**
 - Синтаксический анализ SQL → дерево операций
2. **Переписывание (rewriter)**
 - Разворачивание представлений, CTE, RULE-ов
3. **Планирование (planner)**
 - Перебор всех валидных стратегий
 - Оценка стоимости каждой (в "единицах стоимости")
 - Выбор самого "дешевого"
4. **Исполнение (executor)**
 - Пошаговое выполнение итогового плана

Как оценивается "стоимость"

Каждая операция (фильтр, JOIN, сортировка и т.д.) имеет:

- **startup cost** — стоимость начала операции (например, создания хеша)
- **total cost** — общая стоимость (включая все проходы)

Оценка включает:

- Число чтений/записей с диска
- Затраты на CPU
- Объем обрабатываемых строк
- Потребление памяти

Единицы абстрактны — это не миллисекунды, а **весовые коэффициенты** для сравнения вариантов.

Какие решения принимает планировщик

1. Физический алгоритм доступа

- Seq Scan — построчное сканирование
- Index Scan — использование B-дерева для точечного/диапазонного поиска
- Bitmap Index Scan — оптимизированный способ выборки **многих строк по индексу**, с последующим объединением и чтением данных по страницам

2. Порядок соединения таблиц

- В каком порядке делать JOIN
- Какая таблица внешняя, какая внутренняя

3. Алгоритм соединения (про это подробно в оптимизации)

- Nested Loop
- Hash Join
- Merge Join

4. Когда выполнять фильтры

- До соединения или после?
- Ранний WHERE = меньше данных = дешевле

5. Когда и как делать проекции

- До JOIN → меньше столбцов → быстрее
- После JOIN → дороже

Что помогает планировщику

1. Статистика ANALYZE

- Частота значений
- Количество строк
- Селективность условий

2. Индексы

- Дают альтернативные стратегии доступа

3. Ключи, ограничения

- Помогают исключить невозможные строки

4. LIMIT / OFFSET

- Можно завершить выполнение раньше → выгоднее Nested Loop

5. Настройки памяти

- work_mem, random_page_cost, seq_page_cost

EXPLAIN ANALYZE

Выведет:

- Какой план был выбран
- Сколько строк реально обработано
- Использовался ли индекс
- Где были затраты

Подробно про Физические алгоритмы доступа

Sequential Scan (Seq Scan)

- Чтение **всех страниц таблицы с диска**;
- Каждая строка проверяется вручную;
- **Не зависит от наличия индекса**;
- Используется при:
 - Отсутствии подходящих индексов;
 - Ожидании, что выберется **много строк** (низкая селективность);
 - Очень маленьких таблицах (где быстрее просканировать всё).
- Простой, работает всегда, эффективен для **маленьких таблиц** или **больших выборок**.
- Очень медленно при **выборке малого количества строк** из больших таблиц.

Index Scan

- Происходит **поиск в индексе** (обычно логарифмический).
- Для каждой строки: **индекс → heap (таблица) → visibility check**.
- Позволяет выполнять **диапазонные выборки**.
- Применимо для условий:
 - =, >, <, BETWEEN, LIKE 'abc%'
- Очень быстро на **точечный доступ** (id = 18), эффективно при **небольшом числе подходящих строк**
- При **многих подходящих строках** становится неэффективным, т.к. для каждой строки нужен отдельный **heap access** (чтение строки из таблицы).

Bitmap Index Scan + Bitmap Heap Scan

Используется при

- **много условий в WHERE**, например:
 - a = 10 OR b = 20
 - a IN (1,2,3,...)
- или при **выборке средней/высокой селективности** (например, 10-50% строк)
- Гораздо быстрее Index Scan, если **нужно много строк, объединяет индексы** (поддерживает OR, IN, сложные условия).
- Требуется **дополнительной памяти** под битовую карту, **не работает**, если нет индексированного поля.

Метод доступа	Принцип работы	Идеален для	Когда используется
Sequential Scan	Полный проход по таблице	Все строки, нет индекса	Много строк, нет/плохой индекс
Index Scan	Поиск по индексу → чтение строк	Точная/диапазонная выборка	Малое количество строк, =/<
Bitmap Index Scan	Множественные попадания → битовая карта	Много строк, сложные OR	Условий много, строки разбросаны

Пошаговый разбор

◆ Nested Loop

pgsql

Копировать Редактировать

```
(cost=0.29..283.14 rows=1249 width=17)
(actual time=0.057..2.777 rows=1193 loops=1)
```

- Тип: вложенный цикл соединения — для каждой строки из `Н_СЕССИЯ` — выполнить подзапрос
- Оценка: ~1249 результирующих строк, фактически: 1193
- Время: от начала до конца всей итерации: ~2.7 мс

✓ Подтверждает, что планировщик адекватно предсказал число строк.

1

◆ Seq Scan on "Н_СЕССИЯ"

pgsql

Копировать Редактировать

```
(cost=0.00..117.90 rows=3447 width=8)
(actual time=0.008..0.881 rows=3445 loops=1)
```

- Чтение всей таблицы — **полный проход**
- Фильтрация по дате: `"ДАТА" > '2002-01-04'`
- Удалено строк: 307 (не прошли фильтр)
- Выбрано строк: 3445
- Время: 0.88 мс

✓ Эффективно для объема таблицы, индекс по `"ДАТА"` не использован (вероятно, высокая селективность или отсутствие индекса).

2

◆ Memoize

sql

Копировать Редактировать

```
(actual time=0.000..0.000 rows=0 loops=3445)
```

- Назначение: кэшировать результаты подзапроса `Index Scan`, чтобы одинаковые `"ЧЛВК_ИД"` не приводили к повторным сканированиям.
- Cache Key: `"Н_СЕССИЯ"."ЧЛВК_ИД"`
- Результаты:
 - Hits: 3267 — найдено в кэше ✓
 - Misses: 178 — пришлось выполнить `Index Scan`
 - Evictions: 0, Memory Usage: 16kB — отлично, кэш полностью поместился в RAM

✓ Memoize значительно ускоряет `Nested Loop`, если много повторов по ключу соединения.

5

◆ Index Scan using "ЧЛВК_РК" on "Н_ЛЮДИ"

pgsql

Копировать Редактировать

```
(actual time=0.003..0.003 rows=0 loops=178)
Index Cond: ("ИД" = "Н_СЕССИЯ"."ЧЛВК_ИД")
Filter: ("ОТЧЕСТВО" < 'Владимирович')
Rows Removed by Filter: 1
```

g

- Используется первичный ключ `ЧЛВК_РК`, т.е. `ИД` — индексируется
- Но: дополнительный фильтр по строковому условию: `ОТЧЕСТВО < 'Владимирович'`
- В 178 попытках:
 - `Rows Removed by Filter: 1` — всего 1 раз не прошёл фильтр
 - Остальные — либо не нашли строки, либо прошли

🕒 Итоги:

Параметр	Значение
План выбрал	<code>Seq Scan + Nested Loop + Memoize + Index Scan</code>
Строк обработано	3445 (внешняя таблица)
Итоговых строк	1193
Время исполнения	2.917 мс
Кэш <code>Memoize</code>	Сработал — 96% <code>hits</code>
Использован индекс	✅ <code>ЧЛВК_РК</code>

✅ Заключение: оптимальный план

- PostgreSQL выбрал `Nested Loop + Memoize`, т.к. соединение по `ИД` → очень селективное
- `Memoize` отлично сработал — избежал 3267 повторных `Index Scan`
- `Index Scan` отфильтровал по `ОТЧЕСТВО`
- `Seq Scan` оправдан, если нет индекса по `"ДАТА"` или фильтр не селективен

📌 План идеален для задачи, особенно при таком коротком времени исполнения (2.9 мс).

ПЛАН 2

♦ Верхний уровень: Nested Loop

pgsql

Копировать Редактировать

```
Nested Loop (cost=168.74..1169.19 rows=30 width=14)
(actual time=1.307..7.123 rows=391 loops=1)
Join Filter: ("Н_УЧЕНИКИ"."ВИД_ОБУЧ_ИД" = "Н_ОБУЧЕНИЯ"."ВИД_ОБУЧ_ИД")
Rows Removed by Join Filter: 4
```

- Тип соединения: вложенный цикл (Nested Loop)
- Join Filter: соединение происходит по `ВИД_ОБУЧ_ИД`
- Левый источник — результат Hash Join
- Правый — Index Scan по `Н_ОБУЧЕНИЯ`
- Удалено строк: 4 строки не прошли JOIN FILTER после соединения

✓ PostgreSQL сначала подбирает кандидатов, потом фильтрует по `ВИД_ОБУЧ_ИД`.

♦ Hash Join: `Н_УЧЕНИКИ` ⋈ `Н_ЛЮДИ`

pgsql

Копировать Редактировать

```
Hash Join (cost=168.46..1152.71 rows=31 width=16)
(actual time=1.295..6.206 rows=391 loops=1)
Hash Cond: ("Н_УЧЕНИКИ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
```

- Соединяет: строки из `Н_УЧЕНИКИ` и `Н_ЛЮДИ`
- Строит хеш по `Н_ЛЮДИ` (меньшая таблица)
- Перебирает `Н_УЧЕНИКИ`, ищет соответствие

♦ Ветка 1: Seq Scan по `Н_УЧЕНИКИ`

pgsql

Копировать Редактировать

```
Seq Scan on "Н_УЧЕНИКИ"
(cost=0.00..983.94 rows=117 width=12)
(actual time=0.055..4.702 rows=1468)
Filter: (("ГРУППА")::integer = 1100)
Rows Removed by Filter: 21843
```

- Полный перебор таблицы (~23К строк)
- Оставлено: 1468 строк с `"ГРУППА" = 1100`
- Фильтр селективен (≈6% строк прошло) — разумно

♦ Ветка 2: Bitmap Heap Scan по Н_ЛЮДИ

pgsql  Копировать  Редактировать

```
Bitmap Heap Scan on "Н_ЛЮДИ"  
(cost=34.74..151.60 rows=1349 width=4)  
(actual time=0.495..0.998 rows=1340)  
Recheck Cond: ("ФАМИЛИЯ" < 'Ёлкин')  
Heap Blocks: exact=100
```

♦ Вложенный Bitmap Index Scan:

pgsql  Копировать  Редактировать

```
Bitmap Index Scan on "ФАМ_ЛЮД"  
(cost=0.00..34.40 rows=1349)  
(actual time=0.474..0.475 rows=1340)  
Index Cond: ("ФАМИЛИЯ" < 'Ёлкин')
```

- PostgreSQL применяет **Bitmap Index Scan**, чтобы получить все **ИД** людей, у которых **ФАМИЛИЯ < 'Ёлкин'**
- Потом группирует их по страницам и читает из таблицы
- **Heap Blocks: 100** — реально прочитано 100 страниц таблицы

☒ Отличная оптимизация для массового условия

11

14

♦ Index Scan по Н_ОБУЧЕНИЯ

pgsql  Копировать  Редактировать

```
Index Scan using "ОБУЧ_ЧЛВК_FK_I" on "Н_ОБУЧЕНИЯ"  
(cost=0.28..0.52 rows=1 width=14)  
(actual time=0.002..0.002 rows=1 loops=391)  
Index Cond: ("ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД") AND ("ЧЛВК_ИД" < 163276)
```

- Для каждой из 391 строки после Hash Join выполняется точечный поиск по индексу
- Условие: **ЧЛВК_ИД = ИД** + ограничение **ЧЛВК_ИД < 163276**
- Поиск выполняется быстро — всего 0.002 мс на одну итерацию

16

Итоги исполнения

Этап	Значение
Строк на выходе	391
Удалено строк JOIN фильтром	4
Участвовало строк из <code>н_ученики</code>	1468
Участвовало строк из <code>н_люди</code>	1340
Время исполнения	7.260 мс
Планировщик ожидал	~30 строк, но получил 391
Планируемая сложность	<code>cost=168.74..1169.19</code>

Заключение: план хороший

- **Hash Join** — адекватный выбор для соединения `ученики` и `люди`, т.к. `люди` — меньше, и фильтр `фамилия < 'Ёлкин'` селективен
- **Bitmap Index Scan** — эффективен при множестве подходящих строк
- **Index Scan** по `обучения` — точечный, быстрый
- **Nested Loop + Join Filter** — разумен при небольшом количестве строк (менее 500)